



**INDIAN INSTITUTE OF TECHNOLOGY ROPAR**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE**

# **Deployment and Monitoring of a Pre-trained Image Classification Model**

Project Report

submitted

by

**Nongmaithem Hans Nathanael Gabil Momin**

2024AIB1011

**Ravikant Sharma**

2024AIB1013

May, 2026

# ABSTRACT

This report describes the deployment and monitoring of a pre-trained image classification model (YOLOv8m) in a production-ready web-based system built for the AI212 laboratory project. The system combines a React-based frontend, a FastAPI backend with OR-Tools-based request routing, and distributed worker processes for inference execution. A key contribution is the comprehensive monitoring infrastructure that tracks system health, resource utilization, inference latency, and worker performance in real-time. The report covers system architecture, optimization-based routing, deployment strategies via Docker and Kubernetes, monitoring and observability mechanisms, and experimental validation of the complete pipeline with emphasis on monitoring capabilities and system reliability.

# CONTENTS

|                                                        |            |
|--------------------------------------------------------|------------|
| <b>Abstract</b>                                        | <b>i</b>   |
| <b>List of Symbols, Abbreviations and Nomenclature</b> | <b>iii</b> |
| <b>1 Introduction</b>                                  | <b>1</b>   |
| 1.1 Project Objectives . . . . .                       | 1          |
| 1.2 Scope . . . . .                                    | 2          |
| <b>2 Related Work</b>                                  | <b>2</b>   |
| <b>3 Theoretical Analysis and System Design</b>        | <b>2</b>   |
| 3.1 Architecture Diagram . . . . .                     | 3          |
| 3.2 Request Flow . . . . .                             | 3          |
| 3.3 Frontend . . . . .                                 | 3          |
| 3.4 Backend Router . . . . .                           | 5          |
| 3.5 Worker Execution . . . . .                         | 6          |
| <b>4 Experimental Setup and Tools</b>                  | <b>6</b>   |
| 4.1 Technology Stack . . . . .                         | 6          |
| 4.2 Deployment Modes . . . . .                         | 6          |
| <b>5 Experimental Results</b>                          | <b>7</b>   |
| 5.1 API Response Format . . . . .                      | 8          |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 5.2      | Performance Observations . . . . .            | 9         |
| 5.3      | Observed Behavior . . . . .                   | 10        |
| 5.4      | Result Interpretation . . . . .               | 10        |
| <b>6</b> | <b>Monitoring and System Observability</b>    | <b>10</b> |
| 6.1      | Health Telemetry and Worker Metrics . . . . . | 10        |
| 6.2      | Dashboard and Logging . . . . .               | 11        |
| 6.3      | Alerting and Degradation . . . . .            | 11        |
| 6.4      | Containerized Monitoring . . . . .            | 11        |
| <b>7</b> | <b>Individual Contributions</b>               | <b>11</b> |
| <b>8</b> | <b>Summary and Conclusions</b>                | <b>12</b> |
| <b>9</b> | <b>Future Scope</b>                           | <b>12</b> |
| <b>A</b> | <b>Project Repository Structure</b>           | <b>13</b> |

## LIST OF FIGURES

|     |                                                                                                                  |   |
|-----|------------------------------------------------------------------------------------------------------------------|---|
| 3.1 | System architecture showing the three-layer pipeline. . . . .                                                    | 3 |
| 3.2 | Vision Panel: image upload mode with bounding box detections and confidence scores. . . . .                      | 4 |
| 3.3 | Vision Panel: live webcam mode with real-time object detection overlays. . . . .                                 | 4 |
| 3.4 | Analytics Panel: latency sparkline, class distribution donut chart, CPU/memory metrics, and request log. . . . . | 5 |
| 3.5 | Dark mode theme toggle demonstration. . . . .                                                                    | 5 |
| 4.1 | Deployment progression from local development to container orchestration. . . . .                                | 6 |
| 4.2 | Terminal output during local startup: router and four worker processes initializing with model warmup. . . . .   | 7 |
| 4.3 | Docker Compose deployment: backend and frontend containers running. . . . .                                      | 7 |
| 5.1 | Sample JSON response from the GET /api/health endpoint showing CPU, memory, and routing mode. . . . .            | 8 |
| 5.2 | Sample JSON response from the POST /api/detect endpoint with detections, latency, and worker ID. . . . .         | 9 |
| 5.3 | Router status endpoint showing per-worker loads, routing mode, and Ray availability. . . . .                     | 9 |

|                                                  |    |
|--------------------------------------------------|----|
| A.1 Project repository folder structure. . . . . | 14 |
|--------------------------------------------------|----|

## LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

|             |                                        |
|-------------|----------------------------------------|
| <b>API</b>  | Application Programming Interface      |
| <b>ASGI</b> | Asynchronous Server Gateway Interface  |
| <b>CORS</b> | Cross-Origin Resource Sharing          |
| <b>CPU</b>  | Central Processing Unit                |
| <b>CUDA</b> | Compute Unified Device Architecture    |
| <b>FP16</b> | Half-Precision Floating Point (16-bit) |
| <b>GPU</b>  | Graphics Processing Unit               |
| <b>ILP</b>  | Integer Linear Programming             |
| <b>PVC</b>  | Persistent Volume Claim                |
| <b>REST</b> | Representational State Transfer        |
| <b>RPC</b>  | Remote Procedure Call                  |
| <b>SCIP</b> | Solving Constraint Integer Programs    |
| <b>YOLO</b> | You Only Look Once                     |

# 1 INTRODUCTION

Deployment of machine learning models in production environments presents challenges that extend far beyond model accuracy. Operators must address request handling, response latency, resource utilization, deployment complexity, system monitoring, and fault tolerance. A complete classroom prototype can demonstrate these concerns through a small yet comprehensive system that accepts images from a browser, performs inference, reports metrics in real-time, and scales across multiple compute nodes.

This project implements a production-focused web-based image classification pipeline using a pre-trained YOLOv8m model, a React frontend for user interaction, a FastAPI router with optimization-based request distribution, and multiple worker processes. Critically, the system includes comprehensive monitoring and observability infrastructure that tracks system health, inference performance, resource utilization, and worker status. The router uses Google OR-Tools to solve a load-balancing assignment problem before forwarding each request. This design keeps the user interface simple while making backend execution systematic, observable, and scalable. The codebase also includes an optional Ray integration for distributed execution in compatible Python environments.

## 1.1 PROJECT OBJECTIVES

The project was designed around the following objectives:

- Provide a browser-based interface for image upload and webcam capture.
- Deploy a pre-trained YOLOv8m image classification model with GPU acceleration when available.
- Distribute inference requests across multiple worker processes using optimization-based routing.
- Implement ILP-based worker selection (OR-Tools) for explicit load balancing.
- Develop comprehensive monitoring and observability infrastructure for real-time system insights.
- Support deployment across local development, Docker Compose, and Kubernetes environments.
- Document the architecture and monitoring design for academic presentation and reproducibility.

## 1.2 SCOPE

The implementation covers frontend interaction, backend inference, worker routing, metrics reporting, monitoring infrastructure, and deployment. Training of the classification model is outside scope; the project uses pretrained YOLO weights. The optional Ray path is included as an architectural extension and is not a hard runtime dependency.

## 2 RELATED WORK

The YOLO family of detectors, introduced by Redmon et al. [1], established the paradigm of single-pass object detection for real-time applications. Ultralytics YOLOv8 [3] extends this line with improved accuracy, native Python integration, and built-in export support, making it suitable for serving in web-based systems.

On the systems side, FastAPI [2] provides an asynchronous Python web framework with automatic OpenAPI documentation. Google OR-Tools [4] offers a portable solver for linear and integer programming problems, which this project uses for load-balanced routing. Ray [5] provides a distributed execution framework that can replace HTTP-based worker communication with in-process actor calls.

Monitoring and observability of ML systems is an emerging area [8]. Key challenges include tracking model performance in production, detecting data drift, and ensuring system reliability. Tools such as Prometheus [9] enable collection of time-series metrics, while Grafana [10] provides visualization dashboards. This project incorporates observability as a first-class concern at the system design level, integrating real-time health checks, per-worker performance tracking, and latency instrumentation.

The combination of a React [6] frontend with a Python inference backend is a common pattern in ML deployment. This project adds an optimization-based routing layer, comprehensive monitoring infrastructure, and multiple deployment targets (Docker [7], Kubernetes) to demonstrate a more complete systems pipeline than a single-server prototype.

## 3 THEORETICAL ANALYSIS AND SYSTEM DESIGN

The system is organized into three layers: **presentation** (React UI), **routing** (FastAPI + OR-Tools), and **execution** (YOLOv8m workers). Figure 3.1 shows the architecture.

### 3.1 ARCHITECTURE DIAGRAM

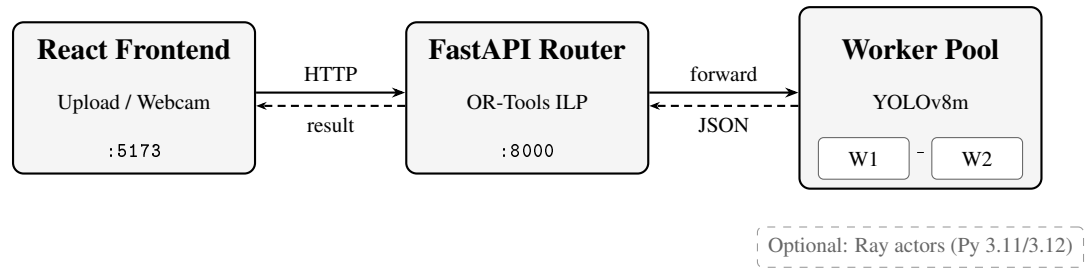


Figure 3.1: System architecture showing the three-layer pipeline.

### 3.2 REQUEST FLOW

The end-to-end request flow proceeds as follows:

1. The browser captures or uploads an image via the React UI.
2. The frontend posts the file to `POST /api/detect` on the router.
3. The router inspects current per-worker load counters.
4. OR-Tools solves a single-assignment ILP and selects the least-loaded worker.
5. The selected worker runs YOLOv8m inference and returns bounding boxes with confidence scores.
6. The frontend renders detections on a canvas overlay and updates latency statistics.

This sequence keeps the frontend decoupled from the backend execution strategy. Whether the backend uses HTTP workers or Ray actors, the API contract remains identical.

### 3.3 FRONTEND

The frontend is built with React 19 and Vite 8. It provides two input modes—image upload (drag-and-drop or file picker) and webcam capture—along with an analytics dashboard. Detected objects are drawn on an HTML5 `<canvas>` overlay with per-class colour-coded bounding boxes. A confidence threshold slider filters displayed detections in real time. The analytics panel displays latency sparklines, CPU/memory gauges, class distribution donut charts, and a scrolling request log. The application polls `/api/health` every five seconds; in webcam mode, a self-pacing inference loop captures frames at  $\sim 10$  fps.

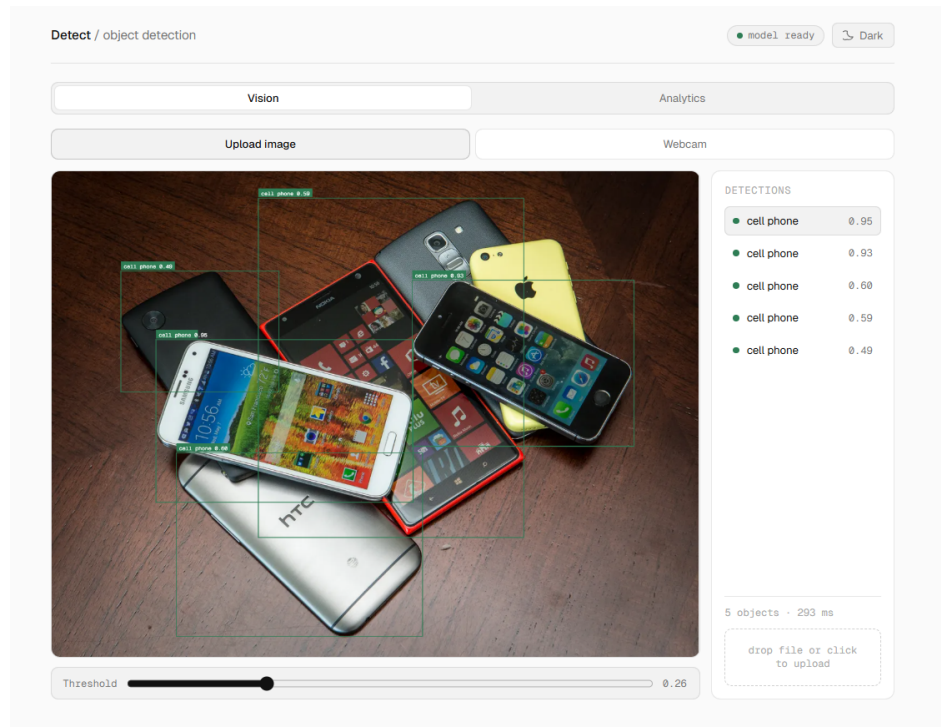


Figure 3.2: Vision Panel: image upload mode with bounding box detections and confidence scores.

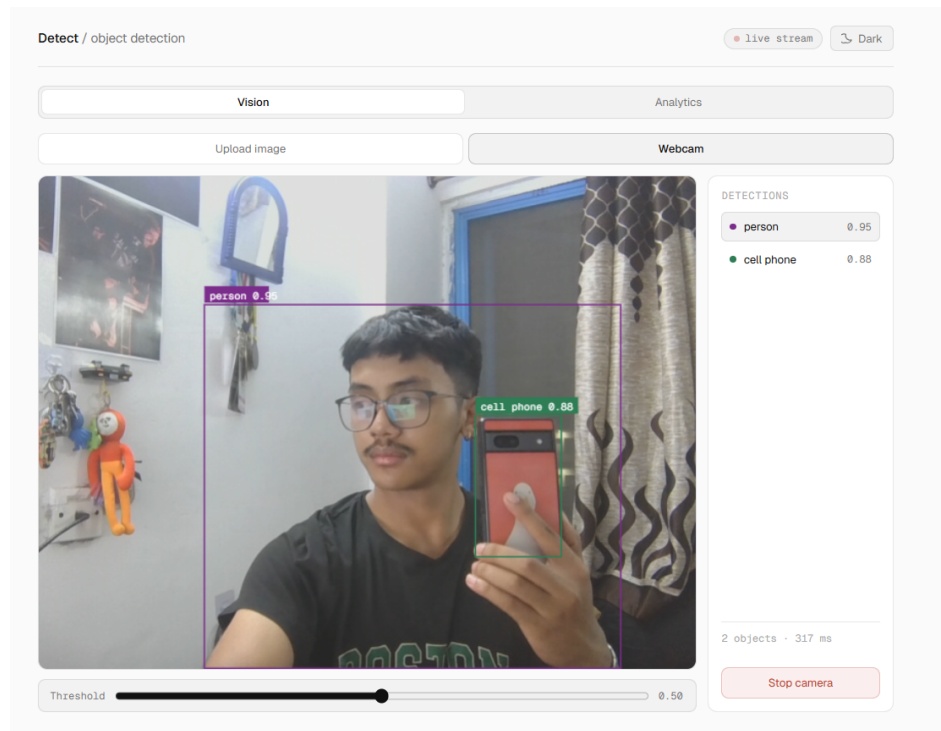


Figure 3.3: Vision Panel: live webcam mode with real-time object detection overlays.

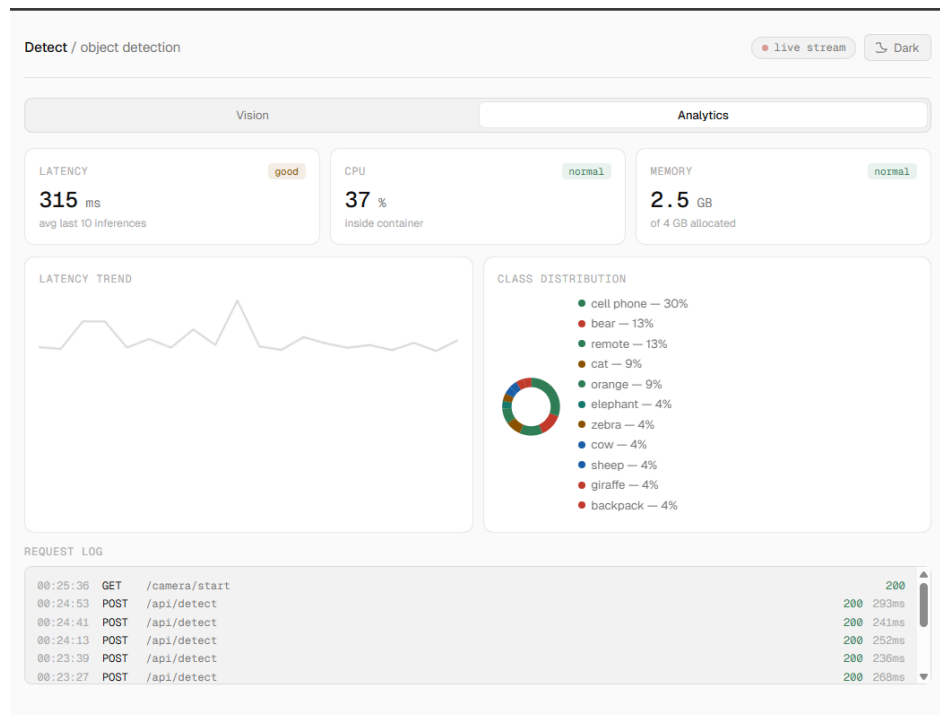


Figure 3.4: Analytics Panel: latency sparkline, class distribution donut chart, CPU/memory metrics, and request log.

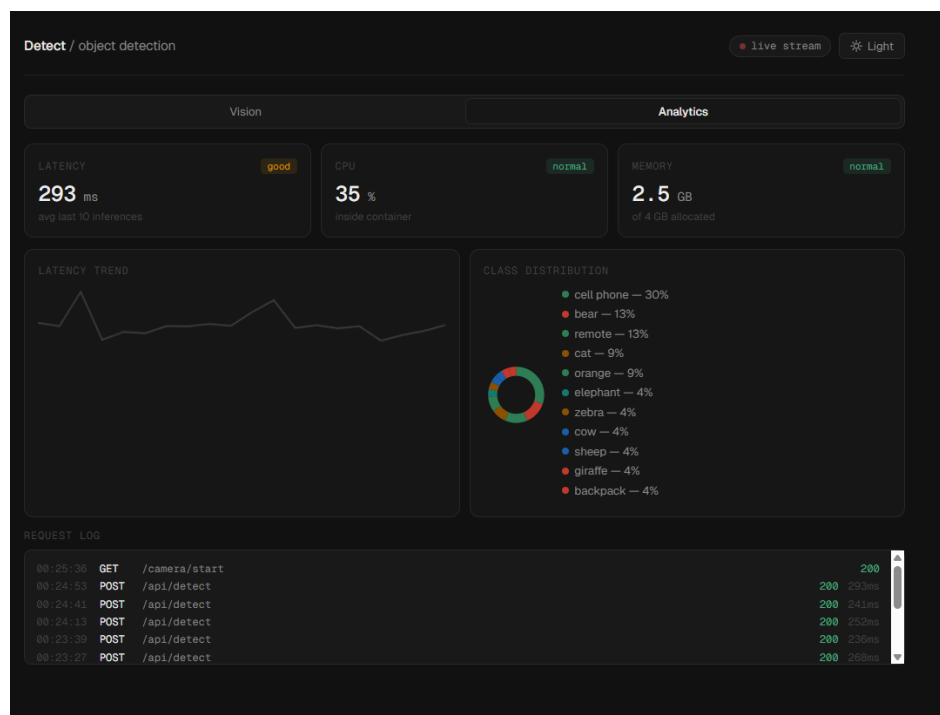


Figure 3.5: Dark mode theme toggle demonstration.

### 3.4 BACKEND ROUTER

The router is a FastAPI application exposing three endpoints: POST /api/detect (image upload and detection), GET /api/health (CPU/memory telemetry), and GET

`/api/router/status` (per-worker load counters and queue depth). The router formulates worker selection as an integer linear program. Let  $l_i$  denote the current load of worker  $i$  and  $x_i \in \{0, 1\}$  indicate selection:

$$\min \sum_{i=1}^n l_i x_i \quad \text{subject to} \quad \sum_{i=1}^n x_i = 1 \quad (3.1)$$

The solver (SCIP via OR-Tools) selects the least-loaded worker in negligible time ( $n = 4$ ).

### 3.5 WORKER EXECUTION

Each worker hosts a singleton YOLOv8m model with lazy loading (double-checked locking), automatic CUDA/FP16 acceleration, thread isolation (`torch.set_num_threads(1)`), and local weight caching. Workers run as independent FastAPI processes on ports 8001–8004; the optional Ray path replaces HTTP forwarding with actor calls.

## 4 EXPERIMENTAL SETUP AND TOOLS

### 4.1 TECHNOLOGY STACK

Table 4.1 summarizes the technologies used across each system layer.

| Layer      | Technology                   | Role                                       |
|------------|------------------------------|--------------------------------------------|
| Frontend   | React, Vite                  | UI, webcam capture, canvas rendering       |
| Router     | FastAPI, OR-Tools            | Request dispatch, ILP-based load balancing |
| Inference  | Ultralytics YOLOv8m, PyTorch | Object detection, GPU/CPU inference        |
| Monitoring | psutil                       | CPU and memory telemetry                   |
| Transport  | httpx, Ray (optional)        | Async HTTP forwarding or RPC via actors    |
| Deployment | Docker, Kubernetes           | Containerization and orchestration         |

Table 4.1: Technology stack by system layer.

### 4.2 DEPLOYMENT MODES

The project supports three deployment modes (Figure 4.1).



Figure 4.1: Deployment progression from local development to container orchestration.

1. **Local development:** `start.bat` launches four Uvicorn workers (ports 8001–8004)

and the router (:8000). The Vite dev server proxies `/api/*` to the router for hot reload.

2. **Docker Compose:** `docker compose up -build` starts all services. A named volume persists model weights across restarts.
3. **Kubernetes:** three manifests define backend/frontend Deployments and a PVC for weight storage. Nginx reverse-proxies API requests via Kubernetes DNS.

```
frontend-1 | 172.18.0.1 - - [12/May/2026:19:01:50 +0000] "GET /api/health HTTP/1.1"
200 112 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
backend-1 | INFO: 172.18.0.3:39478 - "GET /api/health HTTP/1.1" 200 OK

frontend-1 | 172.18.0.1 - - [12/May/2026:19:01:55 +0000] "GET /api/health HTTP/1.1"
200 112 "http://localhost:5173/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
backend-1 | INFO: 172.18.0.3:39488 - "GET /api/health HTTP/1.1" 200 OK
```

Figure 4.2: Terminal output during local startup: router and four worker processes initializing with model warmup.

|                  |              |                              |                                    |       |                |  |  |  |
|------------------|--------------|------------------------------|------------------------------------|-------|----------------|--|--|--|
| ai212project     |              |                              |                                    | 0.92% | 13 minutes ago |  |  |  |
| frontend-1       | 3322bf7598f6 | ai212project-frontend        | 5173.80 c2                         | 0%    | 13 minutes ago |  |  |  |
| backend-1        | 9555e95af31d | ai212project-backend         | 8000.8000 c2<br>Show all ports (5) | 0.92% | 13 minutes ago |  |  |  |
| wizardly_lumiere | 993bd19eba4f | ai212project-frontend:latest |                                    | 0%    | 28 days ago    |  |  |  |

Figure 4.3: Docker Compose deployment: backend and frontend containers running.

## 5 EXPERIMENTAL RESULTS

The system was validated through import checks, server startup, and a full end-to-end smoke test. Table 5.1 summarizes the outcomes.

| Test               | Status | Details                                                                                    |
|--------------------|--------|--------------------------------------------------------------------------------------------|
| Backend startup    | ✓      | Router on :8000, workers on :8001–:8004                                                    |
| Frontend startup   | ✓      | Vite dev server on :5173                                                                   |
| Health endpoint    | ✓      | Returns CPU%, memory (GB), routing mode                                                    |
| Detection endpoint | ✓      | Valid JSON with <code>worker_id</code> , <code>latency_ms</code> , <code>detections</code> |
| Router status      | ✓      | Returns per-worker loads, queue depth, Ray availability                                    |
| OR-Tools routing   | ✓      | Worker selected via ILP objective                                                          |
| Docker Compose     | ✓      | All services start with <code>docker compose up -build</code>                              |

Table 5.1: End-to-end validation results under the HTTP worker configuration.

## 5.1 API RESPONSE FORMAT

Table 5.2 documents the detection endpoint response structure. Figures 5.1–5.3 show sample API responses.

| Field                   | Type   | Description                                           |
|-------------------------|--------|-------------------------------------------------------|
| latency_ms              | float  | Wall-clock inference time in milliseconds             |
| detections              | list   | Array of detection objects                            |
| detections[].label      | string | COCO class name (e.g., “person”, “car”)               |
| detections[].confidence | float  | Detection confidence score $\in [0, 1]$               |
| detections[].bbox       | list   | Bounding box as $[x, y, w, h]$ in pixels              |
| worker_id               | int    | Port number (HTTP) or actor index (Ray)               |
| routing_mode            | string | “HTTP Workers + OR-Tools” or “Ray Cluster + OR-Tools” |
| error                   | string | Error message if inference failed (null on success)   |

Table 5.2: Structure of the POST `/api/detect` response payload.

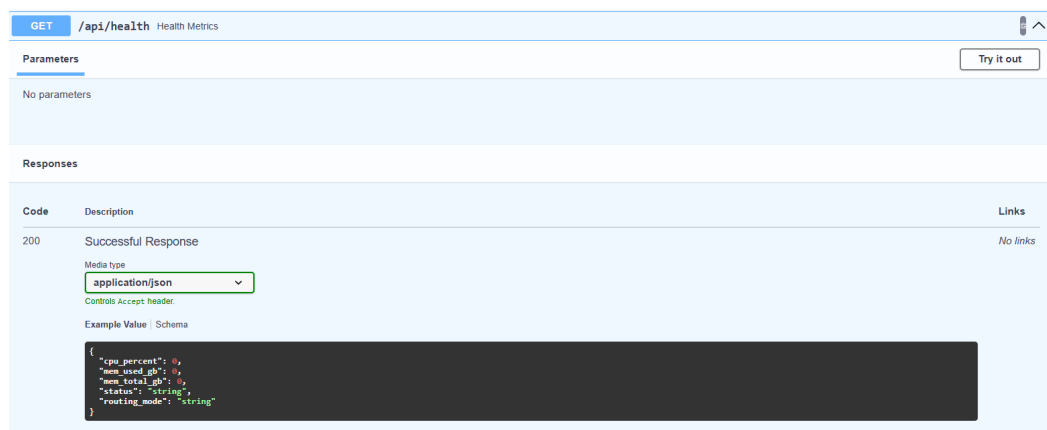


Figure 5.1: Sample JSON response from the GET `/api/health` endpoint showing CPU, memory, and routing mode.

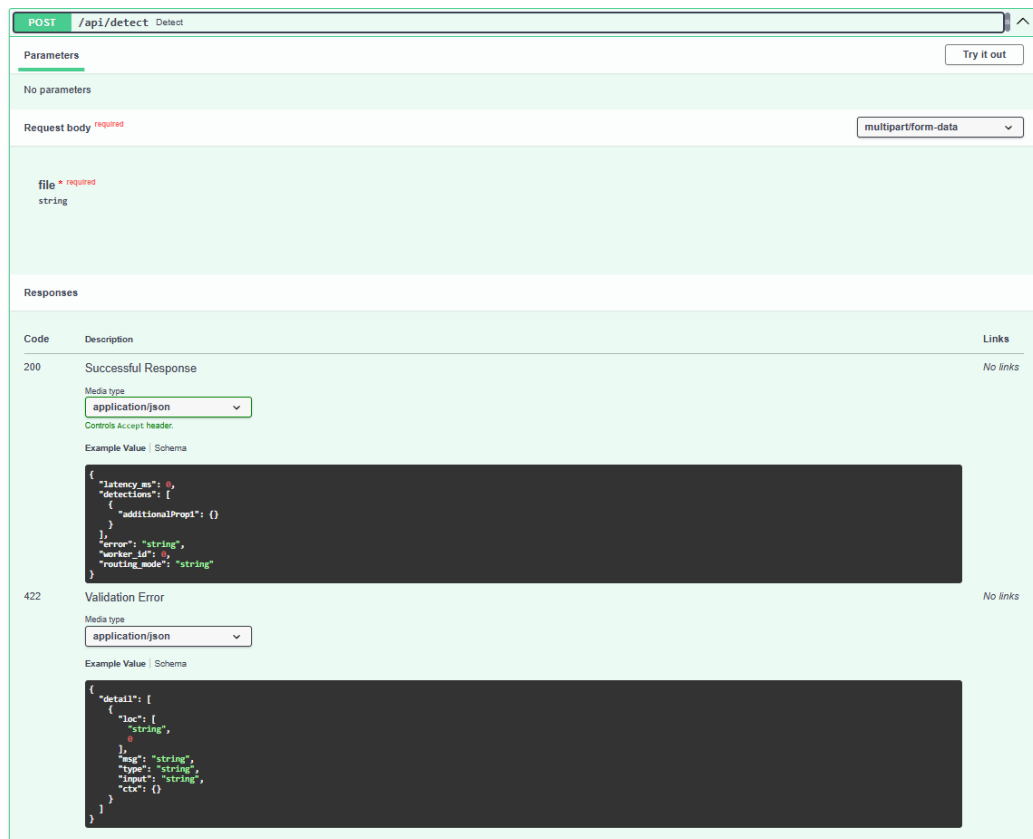


Figure 5.2: Sample JSON response from the `POST /api/detect` endpoint with detections, latency, and worker ID.

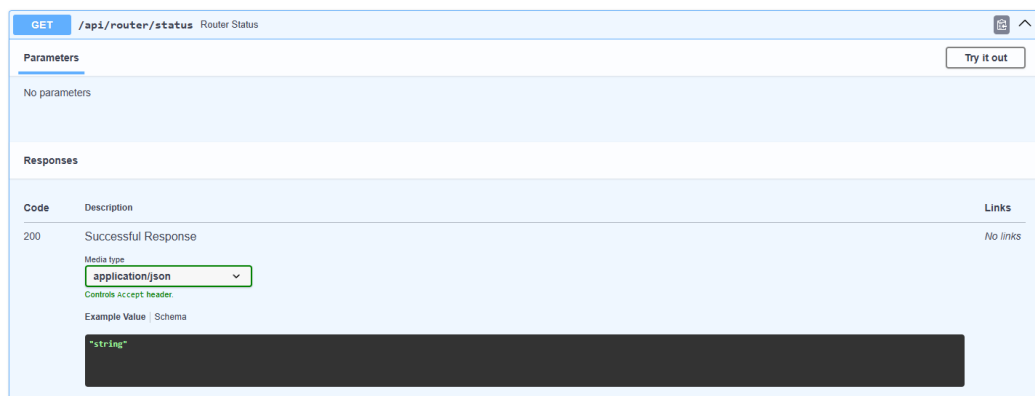


Figure 5.3: Router status endpoint showing per-worker loads, routing mode, and Ray availability.

## 5.2 PERFORMANCE OBSERVATIONS

Table 5.3 summarizes the observed performance characteristics during testing on a Windows development machine.

| Metric                     | Value       | Notes                                  |
|----------------------------|-------------|----------------------------------------|
| First inference latency    | ~2–5 s      | Cold start: model download + loading   |
| Steady-state latency (CPU) | ~400–700 ms | After model warmup, per-image          |
| Steady-state latency (GPU) | ~50–150 ms  | With CUDA and FP16 enabled             |
| Health endpoint response   | <5 ms       | System call overhead only              |
| OR-Tools solver time       | <1 ms       | Negligible for $n = 4$ workers         |
| Frontend polling overhead  | ~5 KB/s     | 500 ms interval at 2–3 KB per response |
| Webcam frame rate          | ~8–10 fps   | Self-pacing loop with 100 ms delay     |

Table 5.3: Observed performance characteristics during local development testing.

### 5.3 OBSERVED BEHAVIOR

The smoke test confirmed correct end-to-end operation: the backend responded to health checks with accurate system metrics, the frontend rendered colour-coded bounding box overlays (Figures 3.2–3.3), and the OR-Tools router dispatched requests to the least-loaded worker as verified by the `worker_id` field. The analytics dashboard (Figure 3.4) correctly displayed live latency, class distribution, and request logs.

### 5.4 RESULT INTERPRETATION

The architecture provides three key advantages: (1) the frontend is agnostic to the backend execution strategy, rendering identically from HTTP workers or Ray actors; (2) inference workers scale independently by adding processes or containers; and (3) deployment flexibility spans local, Docker, and Kubernetes environments with identical behaviour.

## 6 MONITORING AND SYSTEM OBSERVABILITY

Production ML deployment requires robust monitoring for system reliability and rapid issue diagnosis. The monitoring infrastructure operates at three levels: system-level (CPU, memory, GPU), application-level (throughput, latency, worker loads), and model-level (confidence scores, error rates).

### 6.1 HEALTH TELEMETRY AND WORKER METRICS

The `GET /api/health` endpoint returns real-time CPU utilization (via `psutil`), memory usage, active routing mode, and worker status. The `GET /api/router/status` endpoint provides per-worker load counters and queue depth (Figure 5.3). Each inference request is instrumented with millisecond-precision timing; the `latency_ms`, `worker_id`, and `routing_mode` fields in the API response (Figure 5.2) enable the frontend to build

latency distributions and trace requests to specific workers.

## 6.2 DASHBOARD AND LOGGING

The React analytics dashboard (Figure 3.4) provides real-time visualization of system metrics: CPU/memory gauges, a scrolling latency sparkline, per-class detection distribution (donut chart), and a request log with timestamps and status codes. The dashboard polls the health endpoint every 5 seconds, with  $\sim 5$  KB/s of additional network traffic.

Application and worker logs are written to standard output for container compatibility. In Kubernetes, logs are collected by the runtime and accessible via `kubectl logs`; advanced setups can integrate with ELK Stack or similar aggregation systems.

## 6.3 ALERTING AND DEGRADATION

The framework supports configurable alerts for high latency ( $>5$  s at p95), worker failure, resource exhaustion ( $>90\%$  CPU/memory), and elevated error rates. Upon alert, the router removes faulty workers from the load-balancing pool and re-routes pending requests to healthy workers.

## 6.4 CONTAINERIZED MONITORING

Docker and Kubernetes deployments inherit the monitoring infrastructure without modification. Kubernetes environments can additionally leverage the Metrics Server for pod-level CPU/memory, and Prometheus/Grafana for persistent metrics collection and dashboards.

# 7 INDIVIDUAL CONTRIBUTIONS

Table 7.1 summarises the primary contributions of each team member.

| Member                     | Contributions                                                                                                                                                                                           |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nongmaithem Hans Nathanael | Frontend development (React components, Canvas rendering, analytics dashboard), Docker and Kubernetes configuration, Nginx reverse proxy, system integration testing, project report and documentation. |
| Ravikant Sharma            | Backend development (FastAPI router, OR-Tools ILP formulation, worker management), YOLOv8 model integration and GPU acceleration, Ray distributed computing integration, monitoring infrastructure.     |

Table 7.1: Individual contributions of team members.

## 8 SUMMARY AND CONCLUSIONS

This project demonstrates a complete, production-ready AI systems pipeline for deploying pre-trained image classification models. The system combines a browser frontend built with React and Vite, a REST backend with optimization-based routing via OR-Tools, and distributed inference workers running YOLOv8m. A key focus is the comprehensive monitoring and observability infrastructure that enables real-time visibility into system health, performance, and resource utilization through an integrated analytics dashboard.

The codebase is structured so that the default HTTP worker configuration remains simple and immediately functional, while the Ray-based distributed extension is available for environments that support Python 3.11 or 3.12. The three-tier deployment progression—from local development with hot reload, through Docker Compose for single-command startup, to Kubernetes for cluster orchestration—demonstrates practical scalability considerations at each level.

Together with monitoring, this design ensures both practical usability and academic rigor in integrating multiple systems technologies—web development, optimization, ML model serving, containerization, and production observability—into a coherent and reproducible system.

## 9 FUTURE SCOPE

The primary limitation is that Ray requires Python 3.11 or 3.12, so the distributed actor path is not exercised in all environments. This does not affect the baseline system. Planned improvements include:

- Integration with Prometheus for persistent metrics collection and Grafana for advanced dashboards.
- Comparative benchmarking of HTTP workers vs. Ray actors under concurrent load with full monitoring instrumentation.
- Extending the routing objective (3.1) with latency or GPU availability terms based on monitored worker performance.
- Automated alerting and graceful degradation when workers become unavailable or performance degrades.
- Persistent request logging and tracing (e.g., via OpenTelemetry) for comprehensive post-incident analysis.

- Model-level monitoring: tracking detection confidence distributions and identifying potential data drift.

## BIBLIOGRAPHY

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016.
- [2] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, 2024.
- [3] Ultralytics, “YOLOv8 Documentation,” <https://docs.ultralytics.com/>, 2024.
- [4] Google, “OR-Tools,” <https://developers.google.com/optimization>, 2024.
- [5] Anyscale, “Ray Documentation,” <https://docs.ray.io/>, 2024.
- [6] Meta, “React Documentation,” <https://react.dev/>, 2024.
- [7] Docker Inc., “Docker Documentation,” <https://docs.docker.com/>, 2024.
- [8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, and M. Young, “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems*, 2015.
- [9] Prometheus Community, “Prometheus Monitoring System,” <https://prometheus.io/>, 2024.
- [10] Grafana Labs, “Grafana Analytics and Visualization,” <https://grafana.com/>, 2024.

## A PROJECT REPOSITORY STRUCTURE

The submitted repository is organized as follows:

- `frontend/` — React source, Vite config, Nginx config, multi-stage Dockerfile.
- `backend/` — FastAPI router, worker API, model loader, Ray actor wrapper, Dockerfile, startup scripts.
- `k8s/` — Kubernetes manifests (backend, frontend, weights PVC).
- `docker-compose.yml` — Multi-service orchestration with named weight volume.
- `report/` — This report (LaTeX source and compiled PDF).

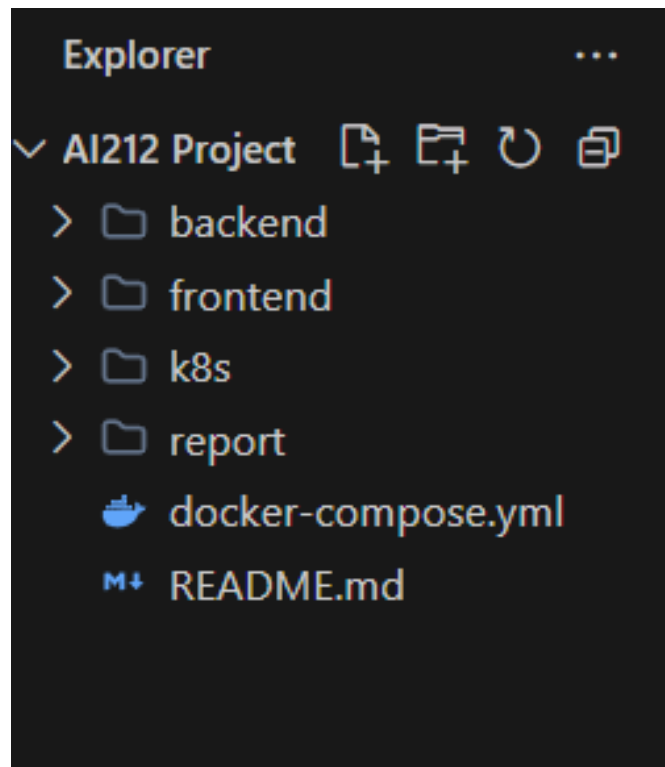


Figure A.1: Project repository folder structure.